SHENZHEN UNIVERSITY

COURSE

Data Analytics in Marketing
Shenzhen University

Chapter 3 — Data Preparation
Cleaning, joining, missing data and data quality

Lecturer: Xieer Dai (David)

Shenzhen University · Weeks 3–14 · Thursday, periods 11–12

Data Analytics in Marketing · course 0205360001

Shenzhen University | Sifang Building South 207 | xieer.dai@szu.edu.cn



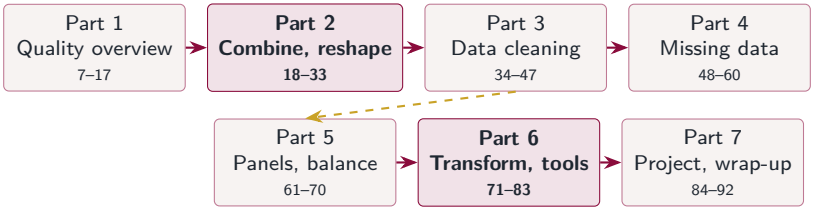
Learning Objectives

After this lecture you should be able to:

1. describe and quantify the quality of a dataset along five characteristics: validity, accuracy, completeness, consistency and uniformity of units;
2. use append / concat (adding observations) and merge / join (adding variables) correctly, work out the join cardinality and verify that the key is unique;
3. move freely between wide and long layouts, and use the reshaping tools pivot / melt / stack / explode / cut fluently;
4. apply the five cleaning steps to duplicates, structural errors, outliers, missing values and validation, and keep an auditable cleaning log;
5. recognise the three missingness mechanisms MCAR / MAR / MNAR, choose and evaluate an imputation strategy, and state what imputation costs;
6. diagnose panel balance (is T constant, where are the gaps), complete the variable transformations and write a reproducible preparation pipeline.

Source: Follows the four strands of the original Lecture 3 deck (Join/Combine/Reshape, Data Cleaning, Missing Data, Tools), extended with 2026 engineering practice.

Roadmap of the Lecture (92 frames)



Spine: raw data → a table you can actually model → reproducible and auditable

10 code frames · 10 sample frames: each page pairs one sample dataset, one figure and one program with line-by-line comments

From Sourcing to Preparation

What you had at the end of L2

- a data-source inventory and re-runnable acquisition scripts;
- several raw files: CSV, JSON returned by APIs, scraped web tables, tables extracted from PDFs;
- but one question was still open: can these data be used directly?

What L3 answers

- how do we quantify data quality? What is missing, what is wrong?
- how are several datasets combined and reshaped into one analysis table?
- how should missing values, outliers and duplicates be handled without leaving hidden hazards?
- how do we make preparation reproducible and auditable?



Practitioner rule of thumb: in an analytics project 60–80% of the time goes into preparation – it is not "dirty work", it is where the credibility of your conclusions comes from.

Deliverables of This Lecture

1) A cleaned dataset

raw / curated layers: the raw snapshot is never touched, the processed layer can be rebuilt (frames 11 and 14)

2) A reproducible script

combine → clean → missing → transform, with assertions and logs (10 code blocks, frames 16–74)

3) A one-page quality report

five-dimension score + quantified findings on missingness, duplicates, outliers and consistency + a treatment list (template on frame 86)

This is Project 1, "Data acquisition and preparation"

The three deliverables of Project 1 are exactly these: a reproducible data pipeline + a data description + a quality report. Frames 84–85 give the requirements and a one-page proposal template (submit by Week 6: question / data / method / expected output).

Source: Marking and submission requirements are in Section 6 of the syllabus; this lecture supplies the report template and the 30-item checklist (frames 86–87).

Map of the Lecture

Part	Frames	Key question
Data quality and preparation	7–17	What counts as "clean"? How do we quantify it? Is the grain right?
Combine and reshape	18–33	Append or merge? Is the key unique? How do we switch wide/long?
Data cleaning	34–47	How do the five steps work? Remove or keep outliers?
Missing data	48–60	Which mechanism is it? Will imputation create artefacts?
Panels and balance	61–70	Is T constant? How do we fill gaps, and should we?
Transformation, tools, engineering	71–83	Transformations and scaling, unit harmonisation, assertions and monitoring?
Project and wrap-up	84–92	What does Project 1 require? What are the common mistakes?

How to read this deck: every part cycles through "concept → sample and visualisation (sample frame) → runnable program (code frame) → checklist".

Why Preparation Dominates the Work

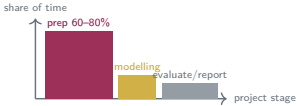
Garbage in, garbage out

Wrong inputs are faithfully amplified by the model: the algorithm raises no error, it returns a plausible-looking wrong conclusion.

The three kinds of preparation work

- Combine: align several datasets into one table;
- Clean: fix errors, remove duplicates, handle outliers and missing values;
- Transform: adjust units, types, distributions and encodings.

Preparation is not "dirty work": it determines how credible your conclusions are, and it is the bulk of the workload in Project 1.



Source: Stance of the "Data Cleaning" section of the original deck ("there is no single set of cleaning steps, but there must be a template") and industry figures on how project time is distributed.

Five Characteristics of Quality

Characteristic	Meaning (original deck wording)	Measurable indicator
Validity	the degree to which the data satisfy defined business rules or constraints	pass rate of regex / range checks, share of parseable dates
Accuracy	the degree to which the data are close to the true value	comparison against an authoritative source, share of impossible values (negative amounts, out of range)
Completeness	the degree to which all required data are known	missing rate; missing by column, by row, in blocks
Consistency	the degree of agreement within one dataset and across datasets	number of category variants, differences in the value of the same entity across sources
Uniform units	the degree to which the data use the same unit of measurement	detection of mixed units, discrepancies after unit conversion

The original deck wrote the fifth item as Uniformity (uniformity of units); the syllabus wording is Uniqueness. Both matter: the first governs "do the same things use the same unit" (developed on frame 73), the second governs "is the primary key duplicated" – this lecture covers both.

Measure quality before fixing it: numbers, not adjectives.

Source: The five characteristics follow page 3 of the original deck; the indicator column is an operational definition added in this lecture.

From Dimension to Action

Characteristic	Detection code (pandas)	Treatment action
Validity	`str.match()`, `to_datetime(errors="coerce")`	repair the format, drop rows that cannot be repaired, add validation rules
Accuracy	range / business-rule assertions, comparison against an authoritative table	re-check the source document, go back to the source system, flag for review
Completeness	`isna().mean()`, clustering of missingness patterns	see Part 4: impute / drop / use methods that tolerate missingness
Consistency	`nunique()`, alias mapping table	build a mapping table and map to a standard code (example on frame 46)
Uniform units	a unit column + conversion functions, dimensional audit	convert to a base unit and state it in the metadata
Uniqueness	`duplicated()`, `merge(validate=...)`	de-duplicate by business rule, keep an audit trail

Principle: quantify first, then decide; every treatment must be able to answer "why was it done and how many rows did it affect".

Source: Detection code and treatment actions compiled from the code frames of this lecture (frames 16, 42, 43, 53).

Grain: What Does One Row Mean

Three questions you must settle first

1. Entity grain: is one row an order, a customer or a store?
2. Time grain: day, month, quarter, or a snapshot date?
3. Aggregation level: raw observations, or already-aggregated totals?

What goes wrong when the grain is wrong

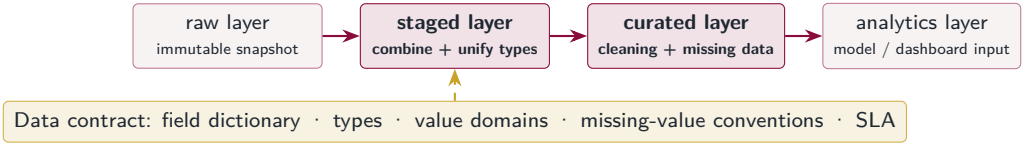
A "customer-month" table treated as a "customer" table → customers are double-counted; a table containing a "total" row treated as detail → the mean is inflated.



one row = customer × month (panel grain)

Source: The engineering extension of the original deck's definitions of cross-section, time series and panel; links to the data-structures section of L2.

Layering and Data Contracts



Why layers

When something goes wrong you can go back one layer and re-run, instead of scraping everything again; every layer can be checked by assertions and responsibilities are clear.

What the contract contains

Field names and meanings, types, units, allowed values, whether missingness is allowed, update frequency and how changes are announced.

Every program in this lecture follows this line: `13_01` produces the dirty raw layer, `13_05` produces the curated analysis table.

Source: Data contracts and layering are standard practice in the modern data stack (they echo "data contracts and data products" in L2).

Clean vs Transform vs Reshape: Keep the Three Apart

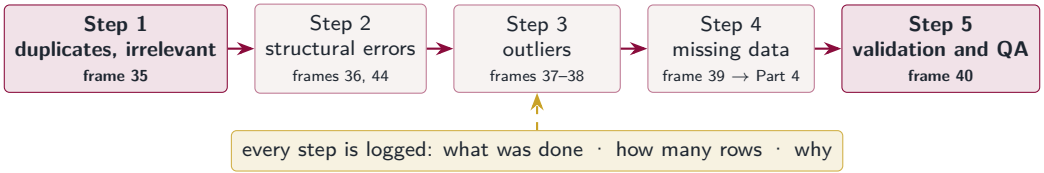
Work	Definition (original deck)	Typical actions and where they appear
Cleaning	removing or repairing data that "does not belong to this dataset"	de-duplication, structural repair, outlier decisions, missing-value treatment (Parts 3–4)
Transformation	changing data from one format or structure into another (wrangling / munging)	type conversion, unit harmonisation, logs and standardisation (Part 6)
Reshaping	changing the layout of the data without changing its content	wide↔long, pivoting, crosstabs, stacking (Part 2)

The three often appear in one script, but they must be logged separately: cleaning changes the facts, transformation does not change the facts, reshaping only re-arranges them.

Cleaning removes what does not belong; transformation changes the form; reshaping changes the layout.

Source: Definitions follow the "Data cleaning vs data transformation" page of the original deck.

The Five Cleaning Steps (the original deck's spine)



In the original deck's own words: there is no set of cleaning steps that fits every dataset – different data need different steps; but you must build a template so that you get it right every time.

A template beats improvisation: same steps, logged, in the same order, every time.

Source: The five steps follow pages 11–15 of the original deck; "log the impact of each treatment" is an audit requirement added in this lecture (implemented in the code on frame 42).

Five Rules for Reproducible Preparation

- Snapshots are immutable: raw files are written to disk by date and never overwritten;
- Script everything: all processing is code; edit something by hand once and it is no longer reproducible;
- Assertions up front: check key uniqueness before a merge, check value domains after cleaning, stop on failure;
- Idempotent writes: re-running gives the same result (de-duplicate + overwrite + fixed seed);
- Auditable logs: every step records row counts, the action and the reason.

How we test this in class

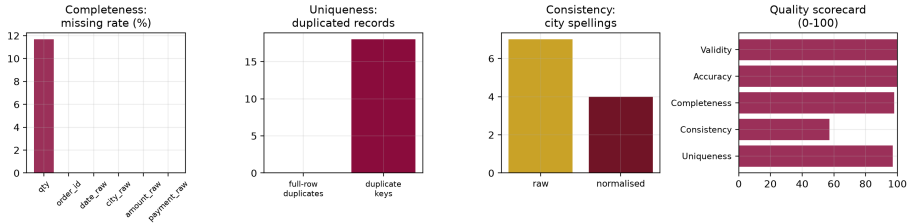
Hand your script + raw data to a classmate with no verbal explanation: do they obtain exactly the same analysis table as you?

Resources for this lecture

fixed seed `SEED = 20260317` · one-command re-run `python run_all.py` · cleaning log `l3_cleaning_log.csv`

Source: Principles compiled from data-engineering practice; they link to "versions and reproducibility" in L2 and to the reporting requirements in L4.

[Sample + visual] Quality Profile Dashboard



Sample `l3_orders_dirty.csv` (600 order rows): 18 duplicate primary keys, 7 spellings of the city, 6 spellings of the payment status, 70 rows with no quantity – all five quality scores in one view.

Program `l3_01_quality_profile.py` (line-by-line comments) · data `website/data/lecture03/` · scorecard `l3_quality_scorecard.csv`

[Code] Profiling a Table

```python
parsed_date = pd.to_datetime(raw["date_raw"], format="mixed", # three source formats, parse what we can
                             dayfirst=True, errors="coerce") # unparseable values become NaT
amount = (raw["amount_raw"]                    # the amount is a signed string
          .str.replace(r"[^\d.\-]", "", regex=True) # keep only digits, the point and the minus sign
          .pipe(pd.to_numeric, errors="coerce")) # unparseable values become NaN
dup_rows = int(raw.duplicated().sum())         # fully duplicated rows (0 in this sample)
dup_keys = int(raw["order_id"].duplicated().sum()) # duplicated business keys (18 planted)
missing_rate = raw.isna().mean().mul(100).round(2) # completeness: missing rate per column
bad_amount = int(((amount <= 0) | (amount > 100000)).sum()) # accuracy: impossible magnitudes
city_raw_n = raw["city_raw"].nunique()         # consistency: variants in the raw column
city_clean_n = raw["city_raw"].str.strip().str.title().nunique() # consistency: variants after cleaning
valid_date_rate = 100 * parsed_date.notna().mean() # validity: share of parseable dates
assert raw["order_id"].str.match(r"^SO\d{7}$").mean() == 1 # fail at once if an id breaks the format
```

Key point: translate "quality" into computable, assertable numbers – this is the first page of the quality report. The full program is in `website/code/lecture03/l3_01_quality_profile.py`.

Summary: Acceptance Criteria for Preparation

Four acceptance criteria

1. Reproducible: a different machine, a different person, the same result;
2. Explainable: every treatment has a reason and a known scope of impact;
3. Traceable: any row of the analysis table can be traced back to the raw snapshot;
4. Verifiable: assertions and a quality report, not eyeballing.

Before Part 2, check yourself

- What does one row represent in my table? Is the grain written down?
- What are my five quality scores, and which is the worst?
- Have I kept an immutable snapshot of the raw data?

Next part (frames 18–33): combining and reshaping several datasets into one table you can analyse.

Append: Adding Observations

The original deck's definition

"Append adds observations to the existing variables." What is appended is an observation; it does not require the two datasets to have exactly the same variables – for example, you have patient data and then receive data on more patients.

In pandas

```
pd.concat([q1, q2], axis=0, ignore_index=True)
```

When column names differ, pandas aligns by name and fills unmatched columns with NaN automatically.

Three must-do actions after appending

1. Align the schema: rename, unify types and units;
2. add a `source` column recording which file each row came from;
3. after concatenating, run a row-count conservation assertion: total = sum over sources.

Same name, different meaning is the most dangerous case: A's `amount` is in yuan and B's is in thousands of yuan, the concatenation shows no error and every conclusion is wrong (see frame 73).

Source: Definition follows the Append page of the original deck (page 4); the engineering actions are compiled from code frame I3_02 of this lecture.

Merge: Adding Variables

The original deck's definition

"Merge adds variables to the existing observations." What is merged are variables; it does not require the two datasets to have exactly the same observations – for example, you have respondent data and then receive answers to the second part of the questionnaire.

In pandas (the original deck's Dataset 1 / 2)

```
d1.merge(d2, on="person", how="inner", validate="1:1")
```

Stata equivalent: `merge 1:1 person using dataset2`

Many rows to one row: `validate="m:1"` (the original deck's `merge m:1 person`)

Three questions you must check

1. Which column is the key, and is it unique?
2. What cardinality do you expect: 1:1 / 1:m / m:1 / m:m?
3. Where did the unmatched rows go? Look at the `indicator` distribution.

Source: Definitions and the Dataset 1/2 example follow pages 5 and 31–33 of the original deck; validate/indicator are pandas' explicit validation tools.

joinby: All Pairwise Combinations within a Group

The original deck's definition

"joinby is similar to merge but forms all combinations of the observations where it makes sense." For example, if table A holds parents and table B holds children, `joinby familyid` produces a dataset of "every parent × each of their children".

Rows from pairing within a group

2 parents × 3 children = 6 rows; summing over all families gives the result row count. Use case: constructing a matched-pairs dataset.

How to do it in pandas, and the risk

```
parents.merge(children, on="family_id", how="inner")
```

Result rows = $\sum$ (number of parents × number of children): the row count is inflated, so before computing anything "per person" be clear about who the denominator is.

Source: The joinby definition and the parent/child example follow page 6 of the original deck; the pandas equivalent and the warning about row inflation are added in this lecture.

Four Join Types and Union

Join	Result (original deck wording)	When there is no match
Inner	keep only values that match in both tables	unmatched values are dropped entirely
Left	keep all values from the left table + matching values from the right	no match on the left → right-hand fields are null
Right	keep all values from the right table + matching values from the left	no match on the right → left-hand fields are null
Full outer	keep all values from both tables	no match on either side → the other side's fields are null
Union	not a join: appending the rows of one table to another	requires the number, names and types of fields to match as far as possible

The practical advice in the original deck (Tableau documentation): if you are not sure which join to use, use relationships – that is, when in doubt do not rush to join: think through the relationships and the grain first.

Row counts are the fastest way to see which join you actually performed.

Source: The five semantics and the wording follow page 7 of the original deck (Tableau's Join Types help page).

[Sample + visual] Join Types on Real Keys

inner join: 5 rows, 0 unmatched

customer_id	order_id	amount
C001	O001	287.58
C001	O002	185.86
C002	O003	146.03
C003	O004	158.19
C003	O005	66.14

left join: 6 rows, 1 unmatched

customer_id	order_id	amount
C001	O001	287.58
C001	O002	185.86
C002	O003	146.03
C003	O004	158.19
C003	O005	66.14
C004	NULL	NULL

right join: 8 rows, 0 unmatched

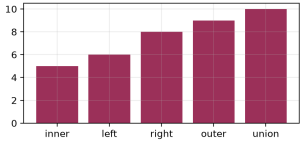
customer_id	order_id	amount
C001	O001	287.58
C001	O002	185.86
C002	O003	146.03
C003	O004	158.19
C003	O005	66.14
C005	O006	402.92

outer join: 9 rows, 1 unmatched

customer_id	order_id	amount
C001	O001	287.58
C001	O002	185.86
C002	O003	146.03
C003	O004	158.19
C003	O005	66.14
C004	NULL	NULL

union: append rows (keeps duplicates)

order_id	customer_id
O001	C001
O002	C001
O003	C002
O004	C003
O005	C003
O006	C005



Left table: 4 customers. Right table: 8 orders (including 3 “orphan” orders and 1 customer with no orders): inner 5 rows · left 6 · right 8 · outer 9 · union 10 – the change in the row count is the evidence of the join semantics.

Program `l3_03_join_types_validate.py` · samples `l3_customers.csv`, `l3_orders.csv` (`website/data/lecture03/`)

Cardinality: Why Row Counts Explode

Keys	Meaning	pandas
1:1	both unique	`validate="1:1"`
1:m	left unique, right repeats	`validate="1:m"`
m:1	left repeats, right unique	`validate="m:1"`
m:m	both may repeat	rows ×: avoid

Course example: the customer table has one duplicated C001 row, so the join turns 5 rows into 8 – without `validate` pandas inflates silently.

Make the error visible

```
try: ... merge(validate="1:1")
except pd.errors.MergeError as err:
  print(err)
```

→ "Merge keys are not unique ... not a one-to-one merge"

The reconciliation (receipt) habit

Record before and after the join: left rows, right rows, result rows and the size of each `indicator` group. Put the four numbers side by side and "silently dropped" or "silently doubled" rows have nowhere to hide.

Source: The cardinality concept and the validate usage follow the pandas documentation (Merge, join, concatenate and compare).

Join Pitfalls

- Duplicate keys: a non-unique key → rows multiply (example on frame 23: 5 rows become 8);
- Null keys: NULL is not equal to anything, so rows with a null key never match;
- Type mismatch: `"001"` and `1` look the same but fail to match;
- Format noise: case, leading and trailing spaces, full-width characters, invisible characters;
- Semantic drift of the key: the same column denotes different entities in different years.

Run this before you join

1. Is the key column unique? `duplicated().sum()`
2. Are the key dtypes identical?
3. Are there null keys, and what share of rows?
4. Normalise: `strip().lower()`, remove full-width characters, unify the encoding;
5. Immediately reconcile by `indicator` group after the join.

The most expensive error is not an error message but an error that raises no message: the row count changes quietly and the model still runs.

Source: Pitfall list compiled from data-engineering practice and the problems that appear in this lecture's samples (l3_02, l3_03).

Key Normalisation and Entity Alignment

Four kinds of normalisation action

1. Form normalisation: trim, case, full-width to half-width, unify the encoding;
2. Alias mapping: `{"Shenzhen City": "Shenzhen"}`;
3. Code-system alignment: administrative divisions, industry codes, ticker suffixes;
4. Entity resolution: renames, parent-child relations, duplicate names $\rightarrow$ an entity master table + a mapping table.

What it does in this lecture's sample

The city column goes from 7 spellings to 2 cities; the payment status from 6 spellings to 4 values (including an explicit `unknown`). The number of variants before and after normalisation is the most direct quantitative measure of consistency.

The mapping table must be versioned (`13_city_names_map.csv`): it determines how you interpret the data, so it is part of the method.

Source: Form normalisation and alias mapping are common practice; the mapping-table example of this lecture is website/data/lecture03/l3_city_names_map.csv.

[Code] append / concat and Schema Alignment

```python
naive = pd.concat([q1, q2], ignore_index=True) # stack the two extracts together directly
print("naive concat shape:", naive.shape) # 500 x 9: renaming produced empty columns
print(naive.isna().sum())                 # every renamed column has become one big hole
RENAME = {"OrderID": "order_id", "store": "store_id", "quantity": "qty"} # old name -> standard
def align(df, source):                    # normalise one extract to the target structure
    out = df.rename(columns=RENAME).copy() # rename first, then work on the copy
    out["date"] = pd.to_datetime(out["date"], format="mixed", dayfirst=True, errors="coerce") # dates
    out["amount"] = pd.to_numeric(        # the amount still carries a currency symbol
        out["amount"].astype(str).str.replace(r"[^\d.\-]", "", regex=True), # strip the symbol
        errors="coerce")                  # unparseable values become NaN
    out["qty"] = pd.to_numeric(out["qty"], errors="coerce").astype("Int64") # nullable integer
    out["order_id"] = out["order_id"].astype("string") # primary key cast to string explicitly
    out["store_id"] = out["store_id"].astype("string") # store column cast to string as well
    out["channel"] = out.get("channel", pd.Series(pd.NA, index=out.index)).astype("string") # fill
    out["source"] = source                # source column: which file this row came from
    return out[["order_id", "date", "store_id", "amount", "qty", "channel", "source"]] # same order
combined = pd.concat([align(q1, "Q1"), align(q2, "Q2")], ignore_index=True) # 500 x 7
assert len(combined) == len(q1) + len(q2), "concat lost or invented rows" # row-count conservation
assert combined["order_id"].is_unique, "duplicate order ids after concat" # primary key uniqueness
```

Compare: the naive concatenation has 9 columns that are all holes, the aligned version has 7 clean columns; the two assertions upgrade "it runs" to "it can be trusted". Full program `13_02_join_concat.py`.

[Code] Safe Merges: validate and indicator

```python
print(dataset1.merge(dataset2, on="person", how="inner", validate="1:1")) # merge 1:1 person
print(dataset3.merge(dataset2, on="person", how="left", validate="m:1")) # merge m:1 person
print(pd.concat([dataset1, dataset4], ignore_index=True)) # append using dataset4
for how in ["inner", "left", "right", "outer"]: # the four joins of the original deck
    res = customers.merge(orders, on="customer_id", how=how, # merge on the business key
                          indicator=True)      # record which side each row came from
    print(how, len(res), res["_merge"].value_counts().to_dict()) # row count and origin breakdown
dup_keys = pd.concat([customers, customers.iloc[[0]]], ignore_index=True) # plant a duplicate key
try:                                          # here we want an error, not silent inflation
    dup_keys.merge(orders, on="customer_id", how="left", validate="1:1") # declare 1:1
except pd.errors.MergeError as err:       # pandas raises when the declaration is violated
    print("validate=1:1 correctly raised:", str(err).splitlines()[0]) # print the first error line
exploded = dup_keys.merge(orders, on="customer_id", how="left") # no validate: rows double
print("rows without validate:", len(exploded), "vs customers:", len(dup_keys)) # 8 vs 5
safe = orders.merge(customers, on="customer_id", how="left", validate="m:1") # many to one
```

The Stata spellings in the original deck (`merge 1:1` / `merge m:1` / `append`) map onto pandas one by one here; the difference is that pandas can declare the cardinality and let a violation raise an error.

Wide ↔ Long Layouts

Two layouts

- Wide: one row per entity, time or category in the columns;
- Long / tidy: one row per observation, variables in the columns.

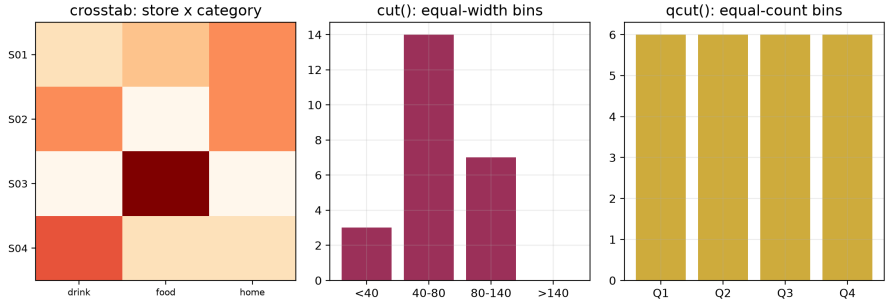
The tidy-data principle: one observation per row, one variable per column. Modelling and plotting libraries usually expect the long layout; reporting outward usually wants the wide layout.

pandas tools and round-trip consistency

`melt` (wide→long) · `pivot` (long→wide) · `wide_to_long` · `stack` / `unstack`
The assertion in this lecture's sample: `back_to_wide` has the same shape as `wide` and every value is equal.

Source: pandas documentation, Reshaping and pivot tables (the topic of page 37 of the original deck).

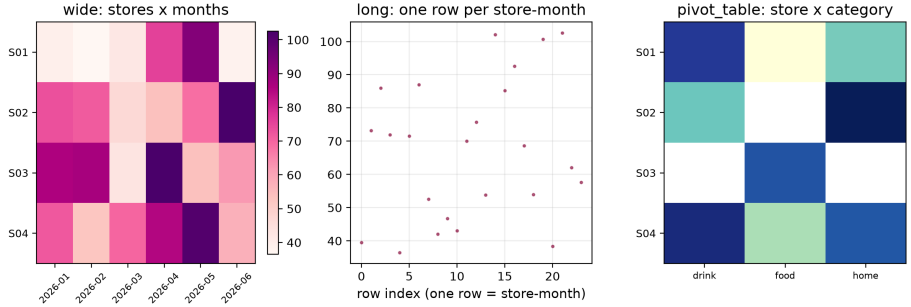
Pivot Tables and Crosstabs



`crosstab` returns frequencies, `pivot_table` returns aggregated values (the mean by default); `cut` bins by equal width and `qcut` bins by equal frequency (6/6/6/6 in this example).

Program `13_04_reshape_toolbox.py` · binning and crosstabs are the two standard ways of turning a continuous variable into categories (candidate model features)

[Sample + visual] One Table, Three Layouts



4 stores × 6 months: the wide table (left) is good for seeing the structure, the long table (middle) for modelling, the pivot table (right) for reporting; the sample assertion verifies the lossless round trip wide→long→wide (the 4×6 shape and every value are identical).

Samples `l3_sales_wide.csv`, `l3_sales_long.csv`, `l3_sales_pivot.csv` (`website/data/lecture03/`)

[Code] The Reshaping Toolbox

```
long = wide.reset_index(names="store_id").melt( # wide -> long: one "store-month" per row
    id_vars="store_id", var_name="month", value_name="sales") # id column, name column, value column
back = long.pivot(index="store_id", columns="month", values="sales") # long -> wide
assert np.allclose(back.values, wide.values), "values changed in the round trip" # lossless
pivot = long.pivot_table(index="store_id", columns="category", # pivot by store x category
                         values="sales", aggfunc="mean").round(1) # the aggregation is the mean
counts = pd.crosstab(long["store_id"], long["category"]) # a crosstab counts frequencies
long["bucket"] = pd.cut(long["sales"], bins=[0, 40, 80, 140, 1000], # equal-width binning
                        labels=["<40", "40-80", "80-140", ">140"]) # readable bin labels
long["quartile"] = pd.qcut(long["sales"], 4) # equal-frequency binning: same count per bin
long["cat_code"] = pd.factorize(long["category"])[0] # categories to integer codes
dummies = pd.get_dummies(long["category"], prefix="cat") # one-hot encoding
restored = pd.from_dummies(dummies).rename(columns=lambda c: "category") # invert the one-hot
stacked = pivot.drop(columns="total").stack() # stack: push columns into rows (one index level)
unstacked = stacked.unstack()            # unstack: restore the original shape
items.explode("items")                   # list-valued cell -> one row per element (3 -> 6 rows)
```

These 11 lines cover the whole toolbox list of the original deck's "Reshaping and pivot tables" page: pivot, pivot_table, stack/unstack, melt/wide_to_long, get_dummies/from_dummies, explode, crosstab, cut, factorize.

Reshaping Pitfalls

- MultiIndex misalignment: after `stack()` the index has one more level; forget `reset_index()` and you get a strange table;
- Duplicate column names: when `pivot` meets more than one row for the same (index, columns) pair it raises an error – aggregate first (`pivot_table`) and then pivot;
- Type degradation: `melt` mixes int and float in one column → the whole column becomes float (the amounts suddenly carry decimals);
- explode creates empty rows: an empty list becomes a NaN row, so check the row count in an assertion;
- Column-order drift: different batches order columns differently, so positional selection breaks – always select by column name.

Three lines of defence

1. Shape assertions: is the relationship between rows and columns before and after reshaping explainable?
2. Value assertions: verify with `np.allclose` after a round trip;
3. Key assertions: is the (index, columns) combination unique after reshaping?

Reshaping does not change the facts, only the layout; if the numbers change after reshaping, some step is wrong.

Source: Pitfalls compiled from the pandas reshaping documentation and the sample implementations of this lecture.

Summary: Reconciliation Checklist

Merge

- is the key unique, is the type consistent, what share of keys are null?
- does the declared cardinality (`validate`) match reality?
- do the `indicator` group counts match expectations?
- result rows = matched rows from both tables + retained unmatched rows.

Append (concat) and reshaping

- are column names, types and units aligned first?
- row-count conservation: total = sum over sources;
- lossless round trips: wide↔long, stack↔unstack;
- are the `source` column and marker columns such as `filled_flag` retained?

Next part (frames 34–47): the five cleaning steps – duplicates, structural errors, outliers, missing values, validation.

What Cleaning Is Worth

The original deck's definition

Data cleaning is the process of repairing or removing erroneous, corrupted, mis-formatted, duplicated and incomplete data. Duplicates and mislabels are most likely when merging several sources: if the data are wrong, neither conclusions nor algorithms are reliable – even when they look right.

The original deck also stresses: there is no fixed set of cleaning steps, but you must build a template for your data so that you get it "right" every time.

Payoff

fewer multi-source errors · higher quality of decision information · explainable definitions · errors that can be located and reviewed

Cost (added in this lecture)

Cleaning changes the facts: deleting rows loses information, filling values invents information. So you must record what was changed, in how many rows, and why.

Source: Definition and payoff follow pages 8–9 of the original deck; "the cost of cleaning" is added from engineering practice.

Step 1: Remove Duplicates and Irrelevant Rows

The original deck's points

- duplicates most often arise during data collection: merging sources, scraping web pages, pulling data from several departments;
- de-duplication is the largest part of cleaning;
- irrelevant observations are rows that do not fit the research question (e.g. studying millennials while other generations are mixed in).

Three kinds of "duplicate" to distinguish

1. Exact duplicates: every field identical → delete;
2. Primary-key duplicates: the same order appears twice (possibly with different amounts) → the business rule for which one to keep must be written down;
3. Business duplicates: same customer, same day, same amount → possibly two genuine orders; do not delete blindly.

This lecture's sample: of 600 rows, 18 primary keys are duplicated; the cleaning log records "how many were deleted and on what grounds".

Source: Points follow page 11 of the original deck; the three-way distinction and the treatment rules are added from engineering practice.

Step 2: Fix Structural Errors

The original deck's example

Measuring or transmitting data produces odd naming, misspellings and inconsistent case, which wrongly splits categories apart: the data contain both `"N/A"` and `"Not Applicable"` even though they should be analysed as the same category.

Common structural errors

leading and trailing spaces · mixed full-width and half-width characters · units inside values (`"1.2 thousand CNY"`) · three date formats at once · booleans written as yes/Y/1

The three-step routine

1. Take stock: use `value_counts()` to see how many variants there are;
2. build a mapping table: raw → canonical (versionable, reproducible);
3. assert: after normalisation the number of categories should equal the expected value.

This lecture's sample: 7 spellings of the city → 2 cities; 6 spellings of the payment status → 4 values.

Source: Points and the original example follow page 12 of the original deck; the three-step routine is added as reproducible practice.

Step 3: Outliers – Remove or Keep

The original deck's position

When a one-off observation "looks like it does not belong": if there is a legitimate reason (say a data-entry error), deleting it helps the data behave; but sometimes the outlier is exactly what proves your theory. Being far from the pack is not the same as being wrong – the purpose of this step is to decide whether the value is valid.

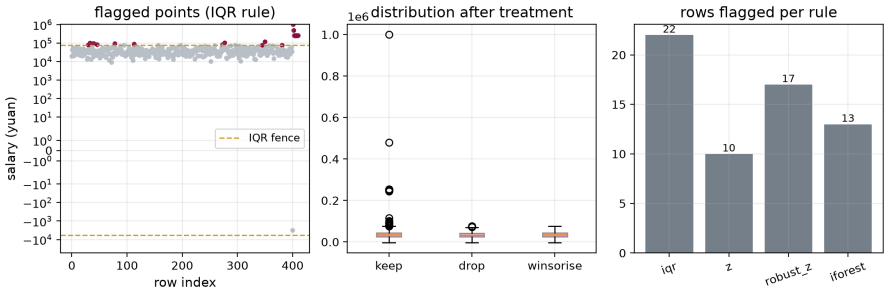
Decision order: 1) Is it an error? (go back to the source document) 2) Is it a real event? (record it) 3) Is it relevant to the research question? – only delete what is "wrong and unrepairable"; a "real but extreme" value should be kept and discussed in the report.

Three treatments (added in this lecture)

- Keep: genuine extreme values (e.g. trading volume on a merger announcement day);
- Winsorise: keep the row but pull the value to a quantile boundary;
- Delete: only when an entry or transmission error is confirmed, recording the count and the reason.

Source: The position and the original example follow page 13 of the original deck; the three treatments and their effect on means and variances appear on frame 38 and code frame 43.

[Sample + visual] Three Outlier Detection Rules



IQR (the box-plot rule) flags 22 rows, Z-score 10, the robust MAD Z 17 and IsolationForest 13; only 10 rows are flagged by all four – so "outlier" depends on the method and the threshold.

Program `13_06_outliers_iqr_z_iforest.py` · sample `13_income_outliers.csv` (2 data-entry errors, 1 genuine extreme value, 8 rows from a special group)

Step 4: Handle Missing Data (leading to Part 4)

The original deck's three options

1. delete observations containing missing values – this loses information, so be clear about the cost before deleting;
2. fill in from other observations – this may import assumptions and loses data integrity;
3. change how the data are used and work around the missingness – use methods that are insensitive to it.

The original deck warns: none of them is optimal; each is a trade-off.

Added in this lecture: answer two questions first

- how much is missing? (by column, by row, in blocks)
- why is it missing? (mechanism: MCAR / MAR / MNAR)

Only once the mechanism is clear can you judge whether deletion introduces bias.

The next part (frames 48–60) is devoted to missingness: mechanisms, three strategies, imputation methods and their cost.

Source: The three options follow page 14 of the original deck; the "ask about the mechanism first" workflow links to Part 4.

Step 5: Validate and QA

The original deck's five questions

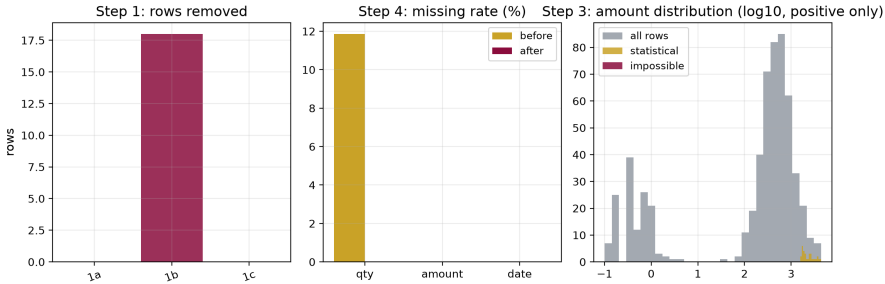
1. Does the data make sense?
2. Does it satisfy the rules for that field?
3. Does it support or refute your hypothesis?
4. Can you see a trend in it that suggests the next hypothesis?
5. If not, is that caused by a data-quality problem?

Executable QA (added in this lecture)

- Assertions: types, uniqueness, value domains, closed sets of categories; stop on failure;
- Reconciliation: compare row counts and distributions before and after cleaning (figure on frame 41);
- Sample re-checks: draw 30 rows at random and check them against the source documents;
- Quality report: write conclusions as "finding + impact + treatment" (template on frame 86).

Source: The five questions follow page 15 of the original deck; the four executable QA actions are compiled from the code frames and the new template in this lecture.

[Sample + visual] Before and After Cleaning



600 rows → 582 rows (duplicates and irrelevant rows); quantity missing in 70 rows → 0 (median 6 filled + a missing-indicator column); the amount distribution stays right-skewed, but impossible values are flagged separately rather than quietly deleted.

Program `l3_05_cleaning_pipeline.py` · audit log `l3_cleaning_log.csv` · result table `l3_orders_clean.csv`

[Code] The Five Steps in Code

```python
raw = pd.read_csv(DAT / "13_orders_dirty.csv", dtype=str) # read the dirty sample (all as strings)
raw["qty"] = pd.to_numeric(raw["qty"], errors="coerce") # turn the quantity column into numbers
df = raw.drop_duplicates().copy()        # step 1a: drop fully duplicated rows
df = df.drop_duplicates(subset="order_id", keep="first") # step 1b: keep one row per primary key
df = df.loc[~df["order_id"].isna()].copy() # step 1c: drop rows with no primary key at all
df["city"] = df["city_raw"].map(normalise_city) # step 2: normalise city names via the alias table
df["payment"] = df["payment_raw"].str.strip().str.lower().map(pay_map).fillna("unknown") # step 2
df["date"] = pd.to_datetime(df["date_raw"], format="mixed", dayfirst=True, errors="coerce") # step 2
impossible = (df["amount"] <= 0) | (df["amount"] > 100000) # step 3: physically impossible
q1, q3 = df["amount"].quantile(.25), df["amount"].quantile(.75) # compute the quartiles
iqr = q3 - q1                            # interquartile range
statistical = (df["amount"] < q1 - 1.5 * iqr) | (df["amount"] > q3 + 1.5 * iqr) # box-plot rule
df["flag_impossible"] = impossible       # flag only, never delete silently
df["flag_outlier"] = statistical & ~impossible # statistically extreme but possibly genuine
df["qty_missing"] = df["qty"].isna().astype(int) # step 4: the missingness itself is information
df["qty"] = df["qty"].fillna(df["qty"].median()) # fill with the median and write it to the log
assert df["order_id"].is_unique, "order_id must be unique after cleaning" # step 5: key unique
assert df["date"].notna().all(), "unparsable dates must be resolved" # step 5: dates valid
record("Step 4 missing handling", len(df), len(df), "qty filled with median") # write the log
```

Every step is written to the audit log (`13_cleaning_log.csv`): step name, rows before and after, how many were removed, and why. Full program `13_05_cleaning_pipeline.py`.

[Code] Outliers: Three Rules, Three Treatments

```
core = RNG.lognormal(mean=10.4, sigma=0.45, size=400) # normal salaries (yuan)
errors = np.array([-3200.0, 999999.0])   # two data-entry errors
extreme = np.array([480000.0])           # one real but extreme salary
group = RNG.normal(loc=250000, scale=4000, size=8) # a small group of special records
df = pd.DataFrame({"salary": np.round(np.concatenate([core, errors, extreme, group]), 2)}) # one column
q1, q3 = df["salary"].quantile([.25, .75]) # quartiles of the observed column
iqr = q3 - q1                            # interquartile range
df["iqr_outlier"] = (df["salary"] < q1 - 1.5 * iqr) | (df["salary"] > q3 + 1.5 * iqr) # box rule
z = (df["salary"] - df["salary"].mean()) / df["salary"].std(ddof=0) # the classic Z score
df["z_outlier"] = z.abs() > 3            # the |z| > 3 rule
mad = (df["salary"] - df["salary"].median()).abs().median() # median absolute deviation
robust_z = 0.6745 * (df["salary"] - df["salary"].median()) / mad # robust Z (suits right skew)
iso = IsolationForest(contamination=0.03, random_state=42) # expect about 3% to be anomalous
df["iforest_outlier"] = iso.fit_predict(df[["salary"]]) == -1 # -1 means flagged as an outlier
drop = df.loc[~df["iqr_outlier"], "salary"] # treatment A: drop the flagged rows
wins = df["salary"].clip(lower=q1 - 1.5 * iqr, upper=q3 + 1.5 * iqr) # treatment B: winsorise
print(summary)                           # compare n, mean, median and standard deviation
```

Key conclusion: the Z-score is hypersensitive to extreme values (both the mean and the standard deviation are contaminated), while the robust MAD Z suits right-skewed data better; rows flagged by all three rules are the most suspicious. Full program `13_06_outliers_iqr_z_iforest.py`.

Cleaning Messy Strings

Common ways strings get dirty

- leading and trailing whitespace and invisible characters (`\u200b`, `\n`);
- mixed full-width and half-width characters typed on a CJK keyboard;
- currency symbols and thousands separators inside numbers (`"CNY 1,234.56"`);
- the unit written into the value (`1.2 thousand CNY`);
- inconsistent placeholders (`N/A`, `-`, `null`, `none`).

Processing order

1. cast to str and remove invisible characters;
2. `str.strip()`, full-width to half-width;
3. use a regex to pull out the numeric part and cast it to a number;
4. put the unit in its own column (value $\times$ unit $=$ base unit);
5. map placeholders to `NA` consistently and record it.

The `amount` column in this lecture's sample mixes four notations at once (`CNY` prefix, `RMB` suffix, thousand CNY, plain numbers) – only after unification can the amounts be compared.

Source: Dirty-data types and the processing order are compiled from data-cleaning practice; the sample is l3_orders_dirty.csv produced by l3_01.

Near Duplicates and Record Linkage

Why exact matching is not enough

"Shenzhen Univ.", "Shenzhen University" and "SZ University", or a company name with and without "Ltd", look like different strings to a computer but are the same entity in business terms.

Common similarity measures

edit distance (Levenshtein) · Jaro–Winkler · hash after standardisation (exact equality) · bag-of-words / embedding similarity

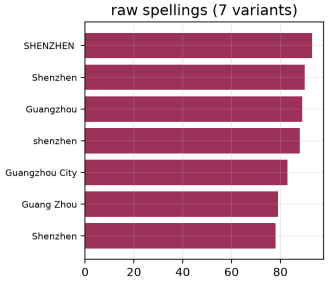
Three practical steps

1. Standardise: strip spaces and punctuation, handle script variants, unify the case;
2. Blocking: compare only records in the same province or with the same prefix, to avoid $O(n^2)$;
3. Human confirmation: re-check a sample above the similarity threshold, then write the confirmed pairs into a mapping table and version it.

Principle: do not guess what a rule can decide; where you must guess, keep the confidence score and the record of human confirmation.

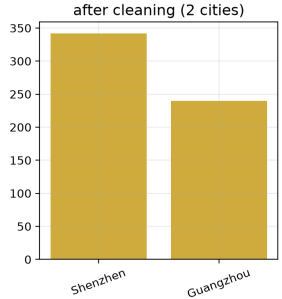
Source: Similarity measures and blocking are standard record-linkage practice; this lecture demonstrates them with the standard-library difflib (see the mapping-table idea on code frame 42).

[Sample + visual] Structural Repair



alias mapping applied

raw	canonical
shenzhen	Shenzhen
shēnzhèn	Shenzhen
guangzhou city	Guangzhou
guang zhou	Guangzhou



Left: the 7 raw spellings of the city. Middle: the alias mapping table (raw → canonical). Right: only 2 cities remain after cleaning. The mapping table is itself a deliverable: it determines how you interpret the data.

Samples `l3_city_names_raw.csv`, `l3_city_names_map.csv` (`website/data/lecture03/`)

Summary: Cleaning Boundaries and Audit

Do not clean away the signal

An outlier may be a real event (a merger, a policy, a pandemic); treating "looks odd" as "wrong" makes a study throw away its most informative observations. Decide whether it is valid first, then decide whether it stays.

What the cleaning log must record

step · rows before and after · number removed or modified · grounds (rule / mapping table / human confirmation) · owner

Three self-checks

1. Can I give a reason for every row I deleted?
2. Is every value I filled in marked (`filled_flag`)?
3. Can someone else reproduce the same table from my script?

Next part (frames 48–60): missing data – mechanisms, strategies, imputation methods and their cost.

Where Missingness Comes From

The original deck's definition

A missing value means that for a given observation, a given variable stores no value at all. Missingness is very common and significantly affects the conclusions you can draw from the data.

Two main sources

- Nonresponse: a particular item, or a whole respondent, provides no information; sensitive items (income, health) are more prone to nonresponse;
- Attrition: in longitudinal studies participants drop out part way through, so later measurements are missing.

Missingness reduces how representative the sample is, which distorts inference about the population – so "how much is missing" and "why it is missing" must always be reported together.

Source: Definition, nonresponse and attrition follow page 16 of the original deck.

MCAR: Missing Completely at Random

Definition (original deck)

The event that makes a data item missing is independent of the observable variables and independent of the unobservable parameters; it happens completely at random.

Consequence

Under MCAR, analysis based on the available data is unbiased – but the original deck warns: truly MCAR data are rare in the real world.

Simulation result in this lecture

After deleting 27.4% of incomes at random, the mean of the available sample is 16908.96 against a true value of 17062.73: a bias of -0.9%, essentially negligible.

A clue for diagnosis: missingness is uncorrelated with every variable → more likely MCAR; but you should still run a preliminary check such as Little's MCAR test.

Source: Definition and consequence follow page 17 of the original deck; the numbers come from the l3_07 simulation (fixed seed) in this lecture.

MAR: Missing at Random

Definition (original deck)

The missingness is not random, but it can be explained by fully observed variables.

Why it can be rescued

The original deck notes that MAR is an assumption that cannot be confirmed by a statistical test and rests only on its substantive plausibility; but if parameters are estimated by something like full-information maximum likelihood (FIML), MAR yields asymptotically unbiased estimates.

Simulation result in this lecture

Make "the more educated you are, the less you want to report income": the missing rate is 24.4% and the bias of the available-sample mean is -14.4% – much worse than MCAR – but education is a complete variable, so a model can correct for it.

In practice: under MAR you can correct with "complete variables + regression / multiple imputation"; remember to state the assumption in the report.

Source: Definition and the FIML remark follow page 17 of the original deck; the numbers come from the l3_07 simulation in this lecture.

MNAR: Missing Not at Random

Definition (original deck)

Neither MAR nor MCAR: the missing value is itself related to the reason it is missing. The original deck's example: men who are more depressed are less willing to complete the depression scale.

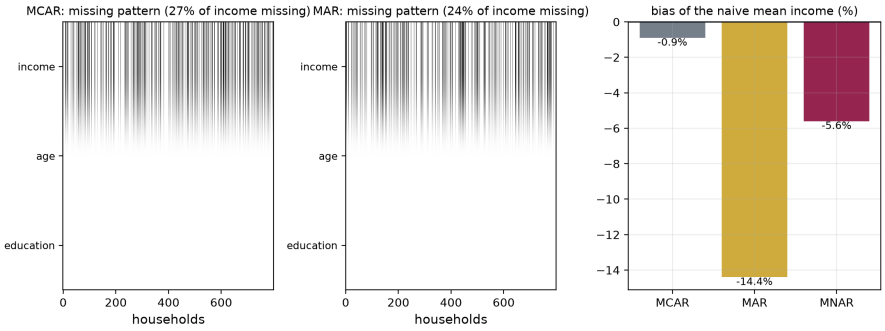
Simulation result in this lecture

Make "the higher the income, the less willing people are to answer": 17.1% missing and a bias of -5.6% in the available-sample mean – note that the size of the bias depends both on the mechanism and on the share of missingness.

MNAR cannot be tested: there is no way to prove it from the data themselves. The only honest response is to run a sensitivity analysis (do the conclusions change under different MNAR assumptions?) and to state the assumption and its limits in the report.

Source: Definition and the original example follow page 17 of the original deck; the numbers come from the l3_07 simulation in this lecture.

[Sample + visual] Missingness Matrix and Bias



Left two panels: the missingness matrices for MCAR (random black dots) and MAR (dots concentrated among the highly educated). Right: the relative bias of the naive mean under the three mechanisms, -0.9% / -14.4% / -5.6%. The missingness pattern shows the mechanism at a glance.

Program `13_07_missing_mechanisms.py` · bias table `13_missing_bias.csv` · three damaged samples `13_income_missing_{MCAR,MAR,MNAR}.csv`

[Code] Simulating MCAR / MAR / MNAR

```python
mu = np.array([base[e] for e in education]) + 0.012 * (age - 22) # income grows with age
income = RNG.lognormal(mean=mu, sigma=0.30) # the complete, known monthly income (yuan)
true_mean = income.mean()                # the quantity we want to estimate
mcar_mask = RNG.random(N) < 0.30         # MCAR: every row has the same 30% chance
mar_p = np.where(education == "master", 0.55, # MAR: missingness depends on education,
                 np.where(education == "bachelor", 0.30, 0.12)) # education is fully observed
mar_mask = RNG.random(N) < mar_p         # draw MAR missingness with those probabilities
mnar_p = np.clip((income - income.min()) / (income.max() - income.min()), 0, 1) ** 2 # richer, quieter
mnar_mask = RNG.random(N) < 0.15 + 0.55 * mnar_p # MNAR: high incomes are more likely to be missing
for name, mask in {"MCAR": mcar_mask, "MAR": mar_mask, "MNAR": mnar_mask}.items(): # three versions
    damaged = complete.copy()            # leave the complete rows untouched
    damaged.loc[mask, "income"] = np.nan # delete only the income values
    naive = damaged["income"].mean()     # estimate the mean from the observed rows
    bias = naive - true_mean             # bias of the naive estimate
    print(name, round(100 * bias / true_mean, 1), "%") # -0.9% / -14.4% / -5.6%
grid.loc[drop_firm | drop_tail, "revenue"] = np.nan # panel sample: a whole block disappears
coverage = grid.groupby("firm_id")["revenue"].apply(lambda s: s.notna().sum()) # coverage
```

One complete dataset, three missingness mechanisms: only under MCAR is the naive estimate unbiased. Full program `13_07_missing_mechanisms.py`.

Three Strategies (original deck)

Strategy	Original deck wording	Key point for this lecture
Imputation	replace missing data with a value that is "filled in"	the method and the reason must be written down first; imputed values must be flagged
Omission	drop samples containing invalid data from later analysis	report how many were dropped and who they were (is a particular kind of case dropped systematically?)
Analysis	use methods that are not affected by the missingness	e.g. the partially overlapping samples t-test (the original deck's example; valid under normality + MCAR)

The original deck notes that a systematic review identified 10 standards for preventing and handling missing data, covering study design, conduct, analysis and reporting – in other words, "how to handle missingness" should be settled when the study design is written.

Source: The division into strategies matches page 18 of the original deck; the key points are added in this lecture.

The Imputation Family

From simple to complex

1. Mean / median: fastest, but it shrinks the variance and weakens correlations;
2. Regression imputation: predict from other variables; keeps the relationships but overstates certainty;
3. KNN imputation: use similar neighbours; suited to numeric data, needs standardisation;
4. The EM algorithm (original deck): iteratively estimate the statistics you would have with complete data; usually it does not fill in values row by row;
5. Multiple imputation MI / MICE: generate several datasets, analyse them separately and combine; it can express the uncertainty of imputation.

EM versus MI (original deck)

With multiple imputation each dataset must be analysed separately, and the statistics are sometimes combined in a fairly elaborate way; EM instead handles missingness at the parameter level rather than filling in item by item.

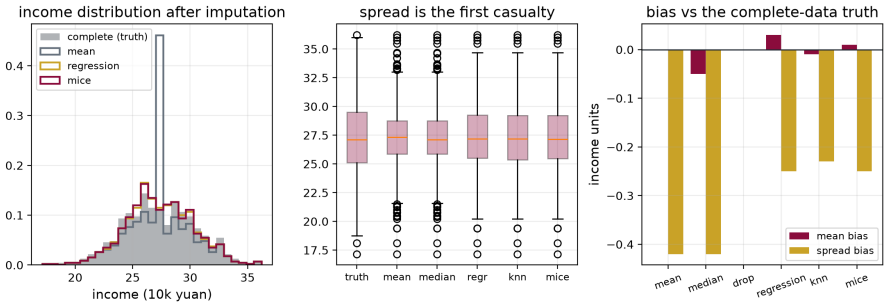
Order of choice: if complete-data or imputation-free methods work, do not impute; when you must impute, prefer multiple imputation and report the sensitivity.

Source: The family corresponds to page 19 of the original deck (Imputation, EM, interpolation); the evaluation criteria are on code frame 56.

[Code] Scoring Imputations

```
missing_idx = damaged["income"].isna()   # rows whose income must be filled
known = damaged.dropna(subset=["income"]) # rows used to fit the model
features = ["hours", "edu", "age"]       # the observable variables available
filled = {"mean": damaged["income"].fillna(damaged["income"].mean()), # mean imputation
          "median": damaged["income"].fillna(damaged["income"].median())} # median imputation
reg = LinearRegression().fit(known[features], known["income"]) # the regression-imputation model
filled["reg"] = damaged["income"].copy() # start from a copy of the observed column
filled["reg"].loc[missing_idx] = reg.predict(damaged.loc[missing_idx, features]) # fill predictions
knn = KNNImputer(n_neighbors=5).fit_transform(damaged[features + ["income"]]) # KNN imputation
filled["knn"] = pd.Series(knn[:, -1], index=damaged.index) # take the value of similar neighbours
mice = IterativeImputer(max_iter=10, random_state=42).fit_transform(damaged[features + ["income"]]) # MICE
filled["mice"] = pd.Series(mice[:, -1], index=damaged.index) # result of the conditional model
for name, series in filled.items():      # compare each strategy with the truth
    print(name, series.mean() - truth_mean, series.std() - truth_sd) # level bias / spread bias
```

[Sample + visual] Imputation Scored



Left: mean imputation squeezes the distribution into a single spike; regression, KNN and MICE stay closer to the true distribution. Middle: the box plots show variance shrinkage. Right: the level and spread bias relative to the true values of the complete data.

Program `l3_08_imputation_compare.py` · metrics table `l3_imputation_metrics.csv` · damaged sample `l3_income_missing.csv`

What Imputation Costs

Four typical side effects

- Variance shrinkage: the filled values resemble each other too closely, so standard errors are underestimated;
- Manufactured relationships: regression imputation makes the correlation between X and Y stronger than it really is;
- False precision: imputed values are treated as observations in later calculations;
- Invisible assumptions: the MAR assumption never reaches the report and readers believe the data are complete.

Four disciplines

1. Flag imputed values (`filled_flag`) so they can be excluded and recalculated at any time;
2. report the missing rate + the mechanism assumed + the method;
3. run a sensitivity analysis: change the method or the assumption and see whether the conclusions change;
4. prefer multiple imputation so that uncertainty enters the standard errors.

The test is not "does the imputation look plausible" but are the downstream conclusions robust.

Source: Side effects and disciplines compiled from the missing-data literature and engineering practice.

Missingness as Signal, Interpolation over Time

The missing indicator

"Income not reported" may itself be a variable: add an `income_missing` column so the model uses both the value and whether it is missing (a practical approach under MNAR).

Time-series interpolation (the original deck's Stata `ipolate`)

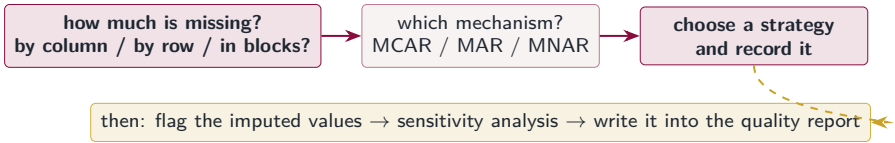
Linear interpolation `ipolate y x, gen(y1);` with extrapolation `..., epolate.` pandas equivalents: `s.interpolate()`, `limit_direction="both"`, `ffill/bfill`.

Interpolation vs extrapolation

- Interpolation: constructing new points between known points; usually reasonable;
- Extrapolation: extending beyond the known range; high risk and it must be flagged;
- time-series interpolation must be grouped by individual (never interpolate across individuals in a panel).

Source: Interpolation and extrapolation follow pages 19 and 27 of the original deck (Stata ipolate/epolate); the indicator variable and the grouping reminder are added in this lecture.

Summary: A Decision Tree for Missing Data



Order of judgement

1) look at the share and the pattern (matrix on frame 52) → 2) determine the mechanism (frames 49–51) → 3) choose a strategy (frame 54) → 4) evaluate with four yardsticks (frame 56)

Three bottom lines

never hide missingness · never treat an imputed value as an observation · never omit the mechanism assumption

Next part (frames 61–70): panels and balance – when "missingness" happens at individual × time cells.

Balanced vs Unbalanced Panels

The original deck's definition

If every cross-sectional observation has the same time periods $t = 1, \ldots, T$, the panel is called balanced; for an unbalanced panel the time dimension is written T_i and differs across individuals.

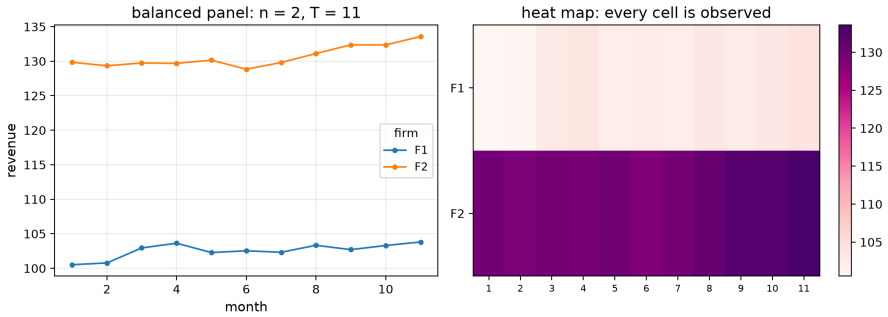
Three common causes of imbalance: individuals entering part way through (a newly listed company), exiting part way through (bankruptcy, delisting), and periods with no observation at all (missing data).

Why it matters

Estimation methods (fixed effects, first differences), the construction of lags and the missingness pattern all depend on the panel structure; if the structure is not diagnosed clearly, the model's assumptions cannot even be discussed.

Source: The definition follows page 20 of the original deck (Section 2 Baseline Definitions).

[Sample + visual] A Balanced Panel $T = 11$, $n = 2$



Left: 11 months of revenue paths for two firms (random walks plus their own level and trend). Right: an individual $\times$ time heat map where every cell has an observation – that is a balanced panel (corresponding to the original deck's "Balanced Panel with T=11 and n=2").

Program `13_09_panel_balance.py` · sample `13_panel_balanced.csv` (`website/data/lecture03/`)

Nominal Balance, Real Gaps

The original deck's two figures

- "Balanced panel with $T = 11$ and $n = 2$ with missing values": the periods are all there but the values are missing;
- "$T_1 = 9,\ T_2 = 11,\ T_3 = 8$ and $n = 3$": the periods themselves differ (unbalanced, or "nominally balanced").

The corresponding sample in this lecture

- structure: T takes $9/11/8$ and $n = 3$;
- values: months 4 and 5 of F2 are blanked on purpose (block missingness);
- coverage table: "observed months / months that should exist" per individual = the coverage rate.

Key distinction: whether the row exists (structure) and whether the value is missing (value) are two different things, handled in two different ways: the first determines the time dimension, the second is missing-data treatment.

Source: The two figures correspond to pages 22–23 of the original deck; the quantitative definition of the coverage rate is added in this lecture.

Why Panels Become Unbalanced

The original deck's Remark

"While the mechanics of the unbalanced case are similar to the balanced case, a careful treatment of the unbalanced case requires a formal description of why the panel may be unbalanced, and the sample selection issues can be somewhat subtle."

The original deck points from here to two issues: sample selection and attrition.

Consequences

- survivorship bias: the individuals that remain are not a random subset;
- an uneven time dimension $\rightarrow$ lag and difference sample sizes change period by period;
- when missingness is correlated with the outcome (the panel version of MNAR), estimates are biased.

So: write down clearly why the panel is unbalanced before deciding whether to fill the gaps – filling blindly hides the sample-selection problem.

Source: The Remark and the quotation follow page 24 of the original deck.

Diagnosing the Panel: Stata `xtset` vs pandas

```
use gymdata, clear          // Stata: load the example panel
list id month wt if id==1 | id==2, sepby(id) // look at two individuals' records
xtset id month              // declare the panel variable and the time variable
```

```
Panel variable: id (unbalanced)
Time variable: month, 1 to 12, but with gaps
```

```
df = df.sort_values(["id", "month"]) # pandas: the same sorting idea
df.groupby("id")["month"].agg(["count", "min", "max"]) # coverage table
df.groupby("id")["month"].apply(lambda s: s.diff().max()) # check the gaps
if df.duplicated(["id", "month"]).any(): # (id, month) must be unique
    raise ValueError("duplicate id-month rows") # raise at once if it is not
```

Stata declares the structure with a single `xtset` and reports "unbalanced / with gaps" automatically; pandas makes you write the coverage table + the gap check + the key-uniqueness assertion yourself – which is exactly what code frame 67 does.

Source: The Stata commands follow page 25 of the original deck; the pandas equivalents are added in this lecture.

Cleaning and Interpolating: Stata against pandas

Stata (pages 26–27 of the original deck)	pandas / Python equivalent
`drop if id==3`	`df = df[df["id"] != 3]`
`keep if year>=2015`	`df = df[df["year"] >= 2015]`
`gen X = wt*2`	`df["X"] = df["wt"] * 2`
`replace X = . if X<0`	`df.loc[df["X"] < 0, "X"] = np.nan`
`order X, before(wt)`	`df = df.reindex(columns=[...])`
`label var X "weight"`	column name / field dictionary (part of the data contract)
`by id: ipolate X t, gen(X_modify)`	`df.groupby("id")["X"].transform(lambda s: s.interpolate())`
`..., epolate`	`interpolate(limit_direction="both")` and flag the extrapolated values

The original deck stresses the linear interpolation of `ipolate` and the extrapolation of `epolate`; engineering adds one more rule: interpolate within each individual, and add a `filled_flag` to the result.

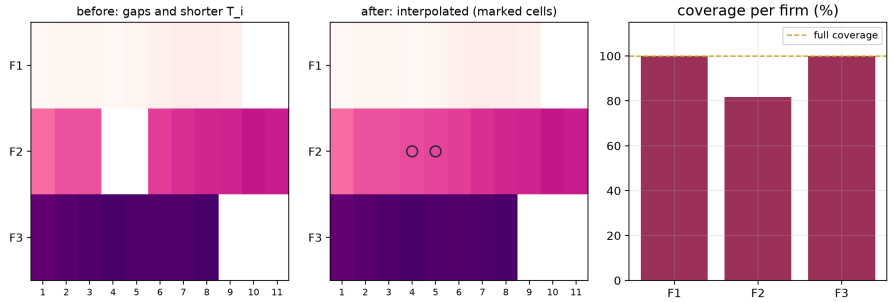
Source: The Stata command list follows pages 26–30 of the original deck; the pandas equivalents are added in this lecture.

[Code] Panel Health Check

```
spec = {"F1": 9, "F2": 11, "F3": 8}      # T1=9, T2=11, T3=8, n=3 (unbalanced to start with)
unbalanced.loc[(unbalanced["firm_id"] == "F2") & # plant one more block of missingness
               (unbalanced["month"].isin([4, 5])), "revenue"] = np.nan # F2 has no months 4, 5
coverage = (unbalanced.assign(observed=unbalanced["revenue"].notna()) # per-firm coverage
            .groupby("firm_id")["observed"].agg(periods="count", observed="sum")) # should be / is
coverage["T_i"] = [spec[f] for f in coverage.index] # the declared number of periods per firm
coverage["coverage_pct"] = (100 * coverage["observed"] / coverage["periods"]).round(1) # coverage
coverage["balanced"] = coverage["periods"] == coverage["periods"].max() # equal period counts?
repaired = unbalanced.sort_values(["firm_id", "month"]).copy() # sort before interpolating
repaired["revenue_interpolated"] = (repaired.groupby("firm_id")["revenue"] # interpolate per firm
                                    .transform(lambda s: s.interpolate(limit_direction="both"))) # fill
repaired["filled_flag"] = repaired["revenue"].isna() & repaired["revenue_interpolated"].notna() # flag
repaired["revenue"] = repaired["revenue_interpolated"] # promote the filled values to the main column
wide = repaired.pivot(index="firm_id", columns="month", values="revenue") # the repaired matrix
print("cells still missing:", int(wide.isna().sum().sum())) # 5: interpolation cannot invent periods
print("imputed cells:", int(repaired["filled_flag"].sum())) # 2 cells were actually filled
```

Teaching point: interpolation can only fill rows that exist; periods that simply do not exist (months 10–11 of F1, months 9–11 of F3) will not be created – filling those requires an explicit decision (add the blank rows + label their origin).

[Sample + visual] Panel Before and After



Left: gaps and a shorter T_i (white cells = no observation). Middle: after per-individual linear interpolation, the imputed cells are marked with hollow circles. Right: the coverage rate of each individual (T_i differs, and so does the coverage).

Program `13_09_panel_balance.py` · samples `13_panel_unbalanced.csv`, `13_panel_repaired.csv`, coverage table `13_panel_coverage.csv`

Case: China's Prefectural Population Panel

The original deck's case

Panel preparation is demonstrated end to end with Chinese prefecture-level city population data (page 28 of the original deck, "An Example").

What you meet in the real world

- Changes in administrative divisions: county-to-district conversion, merging of prefectures, renamings → the definition of the individual changes;
- Changes in definition: permanent resident population vs registered population, revisions of the statistical yearbook;
- Incomplete coverage: some years or some cities have no data (especially the early years).

Processing order

1. build a city master table (with its history and a code mapping);
2. align year by year (using administrative division codes rather than city names);
3. mark the years when the definition changed and model in segments if necessary;
4. deliver a coverage table + a missingness-pattern figure together with the data.

Source: The case follows page 28 of the original deck; the points on definitions and administrative divisions are compiled from experience with statistical yearbooks and link to the data-sources section of L2.

Summary: Panel Preparation Checklist

Structure

- is the primary key `(id, time)` unique?
- is the time axis continuous, and where and how big are the gaps?
- what is T_i for each individual, and is the panel balanced or unbalanced?
- what are the reasons for individuals entering or leaving the sample?

Values and traceability

- is the missingness "a row that does not exist" or "a value that is missing"?
- are the interpolated or filled cells flagged?
- are the coverage table and the missingness matrix delivered with the data?
- is the balance statement written into the report?

Next part (frames 71–83): transformation, tools and engineering – units, distributions and encodings, and turning preparation into a pipeline you can monitor.

Variable Transformations

Four kinds of transformation

- Logarithm: $\log(1 + x)$ compresses the right tail and the coefficient can be read approximately as an elasticity;
- Standardisation (z-score): mean 0, standard deviation 1, comparable across variables;
- Min–max normalisation: maps to $[0, 1]$ and is strongly influenced by extremes;
- Binning: `cut` (equal width) and `qcut` (equal frequency) turn a continuous variable into categories.

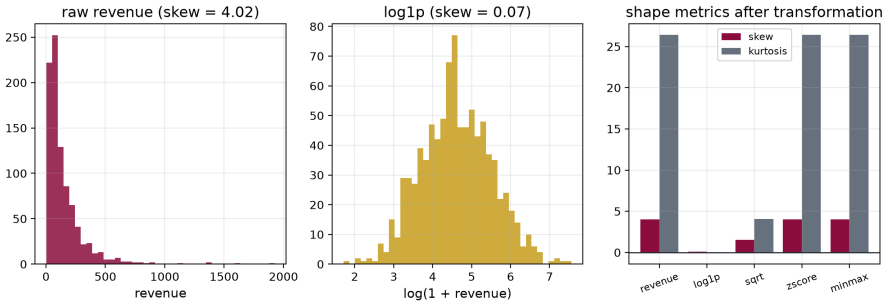
How to choose

1. the shape of the distribution (skewness): clearly right-skewed $\rightarrow$ try a logarithm first;
2. are the scales comparable: several variables entering a model $\rightarrow$ standardise;
3. do you need interpretable coefficients: a log transform beats standardisation;
4. are there genuine extreme values: treat the outliers first, then decide on the transformation.

Remember: a transformation changes the meaning of the variable, so the report must say "the dependent variable is log income".

Source: The transformation list is common practice; the changes in skewness and kurtosis in this lecture's sample are shown on frame 72.

[Sample + visual] Taming Skew



Left: raw revenue (skewness ≈ 2.7, strongly right-skewed). Middle: after $\log(1+x)$ it is nearly symmetric. Right: skewness and kurtosis after each transformation – binning and standardisation do not change the shape of the distribution, only its scale.

Program `l3_10_transform_assert.py` · sample `l3_income_skewed.csv` · summary table `l3_transform_summary.csv`

Uniformity of Units

The original deck's fifth quality characteristic

Uniformity: the degree to which the data are expressed in the same unit of measurement. When units are not uniform, "1.2" may mean yuan, thousands of yuan or millions of yuan.

Common unit traps

yuan / ten-thousand yuan / hundred-million yuan · USD / CNY (and you must fix the exchange-rate convention) · tonne / kilogram · percent vs percentage point · nominal vs real (whether inflation has been removed)

Engineering practice

1. put the unit in its own column and keep the value numeric;
2. define a base unit and conversion functions (in the code, not in your head);
3. assert that after conversion the same entity differs across sources only within tolerance;
4. write the unit, the definition and the exchange-rate source into the field dictionary.

Alongside uniqueness: this lecture brings both the original deck's Uniformity and the syllabus's uniqueness into the quality framework (frames 8–9).

Source: The wording of Uniformity follows page 3 of the original deck; the engineering practice is added in this lecture.

[Code] Transform and Validate

```python
transformed["log1p"] = np.log1p(transformed["revenue"]) # log(1+x) compresses the right tail
transformed["sqrt"] = np.sqrt(transformed["revenue"]) # square root: a gentler compression
transformed["zscore"] = ((transformed["revenue"] - transformed["revenue"].mean()) # centre
                         / transformed["revenue"].std(ddof=0)) # then scale: mean 0, sd 1
transformed["quartile"] = pd.qcut(transformed["revenue"], 4, # equal-frequency binning
                                  labels=["Q1", "Q2", "Q3", "Q4"]) # readable bin labels
summary = pd.DataFrame([{"variable": n, "skew": round(transformed[n].skew(), 3)} # shape metrics
                        for n in ["revenue", "log1p", "sqrt", "zscore", "minmax"]]) # five columns
RULES = {"firm_id": {"unique": True, "regex": r"^F\d{4}$"}, # primary key: unique and well formed
         "region": {"allowed": ["East", "Central", "West"]}, # categories: a closed value domain
         "revenue": {"min": 0.0, "max": 1_000_000.0, "nullable": False}} # a numeric range contract
report = check(transformed, RULES)       # run the contract and collect the violations
print("schema checks passed:", int(report["ok"].sum()), "of", len(report)) # all 12 pass
assert report["ok"].all(), "schema contract failed: fix the table before publishing" # stop
```

This is the "minimum viable data contract": field presence, uniqueness, regex, value domain and non-nullness asserted one by one, and stop on failure – in production the equivalent tools are `pandera` and Great Expectations (frame 78).

Stata Commands → pandas (page 30 of the original deck)

Stata command (original deck)	pandas equivalent
`drop ... if ...` / `keep ... if ...`	`df = df[cond]` / `df = df[~cond]`
`drop(keep) ... if ...` **(select a subset)**	`query()` or boolean indexing
`replace ... if ...`	`df.loc[cond, "col"] = value`
`gen(egen) ... if ...`	`df["new"] = ...` / `groupby().transform()`
`order ... , before(last)`	`df.reindex(columns=[...])`
`label variable ...`	field dictionary / data contract (the column name is the label)
`rename ...`	`df.rename(columns={...})`
`merge 1:1 / m:1 ... using ...`	`merge(validate="1:1" / "m:1")`
`append using ...`	`pd.concat([...])`
`joinby ...`	`merge(on=group)` (row counts multiply: use with care)
`xtset id t`	`df.groupby("id")["t"]` coverage check
`ipolate ... , epolate`	`interpolate(limit_direction="both")`

Source: The command list follows page 30 of the original deck (Stata Commends); the pandas equivalents are added in this lecture, and every code frame from 42 to 74 can be done in Python.

pandas Quick Reference (pages 29, 35–37 of the original deck)

Combining and comparing

`merge` · `join` · `concat` · `compare` · `combine_first` · `update`; pandas provides set logic (union / intersection) for indexes and relational algebra.

Reshaping and pivoting

`pivot` · `pivot_table` · `stack/unstack` · `melt` · `wide_to_long` · `get_dummies/from_dummies` · `explode` · `crosstab` · `cut` · `factorize`

The resources the original deck pointed to

- the official pandas *10 Minutes to pandas* introduction;
- Visual Python: build pandas code through a graphical interface;
- a hands-on tutorial written in Chinese (Zhihu column).

This lecture upgrades the original deck's resource pointers into runnable samples: 10 programs + 39 sample datasets, all with line-by-line comments.

Source: The resource links and the topics follow page 29 of the original deck and the Merge/Reshape chapters of the pandas documentation.

Handling Bigger Tables: Polars and DuckDB

Polars

A columnar memory layout plus lazy execution (LazyFrame): write the whole transformation plan first, then optimise and execute it in one pass; multi-threaded, low memory footprint and an API close to pandas (`select/filter/group_by/join`).

DuckDB

An embedded columnar database: write SQL directly against CSV/Parquet with no import step; suited to teams that prepare data in SQL and to aggregating very large files.

When to switch

- if the data fit in memory (the scale of this lecture's samples) → pandas is enough;
- if single-machine memory is tight and you have hundreds of millions of rows → Polars / DuckDB;
- if you need collaboration, scheduling and lineage → move to a warehouse / lakehouse (frame 79).

Principle: make the pipeline correct first, then make it fast; writing Parquet saves space and preserves types.

Source: The positioning of Polars and DuckDB follows their official documentation (2026); it echoes the modern data stack in L1 and the open table formats in L2.

Reproducible Pipelines: pandera and Great Expectations

pandera

Describe a schema declaratively in Python: column names, types, value domains, uniqueness, closed sets of categories, cross-column rules; it integrates seamlessly with pandas/Polars and can validate automatically at a function's entry point.

Great Expectations

"Expectation suite" style validation: out-of-the-box rules such as `expect_column_values_to_not_be_null`, generating data documentation (Data Docs) that can run periodically inside CI or a scheduler.

How to put it to work

1. keep the contract in the code repository (versioned with the data);
2. validate before every write and stop and alert on failure;
3. archive the validation results as evidence for the quality report;
4. changes to the contract go through review like any other code.

The 20-line assertion pipeline on code frame 74 of this lecture is the smallest kernel of these two tools.

Source: The official pandera and Great Expectations documentation (2026).

dbt and Feature Stores: From Scripts to Platforms

dbt: engineering transformation

One SQL file per model (`select` is the definition) with automatic dependency ordering; built-in tests (unique, not null, relationships), documentation and a lineage graph; incremental materialisation and snapshots.

Feature Store

Solves the inconsistency between the features used for training and the features used at serving time (training–serving skew): a shared feature definition + offline table + online table so that both sides reuse one definition.

How this connects to the lecture

The "layers + contracts + assertions" of this lecture are the smallest kernel of both kinds of platform; a platform helps you scale, but the definitions and the quality judgements are still made by people.

Trade-off: at course-project scale, scripts + contracts + assertions are enough; you only need a platform when several people collaborate and things change often.

Source: Official documentation for dbt and feature stores such as Feast (2026); it links to the modern data stack in L1 and to MLOps in L5.

LLM-Assisted Cleaning (new in 2026)

Where an LLM helps

- Merging long-tail categories: thousands of job titles or industry names → standard categories;
- standardising addresses and company names, "abbreviation → full name";
- extracting from unstructured text: turning a notes field into structured columns;
- explaining anomalies: proposing plausible causes for suspicious records.

Three disciplines

1. Rules first: if a mapping table can solve it, do not use a model;
2. human verification on a sample: draw 30–50 records at random and compare the agreement rate with human labels;
3. archive the prompts and the model: the model version, prompt, temperature and run date go into the report.

Risk warning: LLMs confidently fabricate normalised results (hallucination); for fields such as amounts, codes and IDs, do not let the model "tidy them up in passing". It links to LLM extraction in L2 and to the large-model content in L11.

Source: LLM-assisted cleaning practice and risks compiled from 2025–2026 practice and the course's AI-use policy.

De-identification and Privacy Enhancement

Identifying and handling PII

names, national ID numbers, phone numbers, e-mail addresses, addresses, bank cards, device IDs $\rightarrow$ identify $\rightarrow$ handle $\rightarrow$ record. Ways to handle: deletion, pseudonymisation (salted hashing: linkable but not reversible), generalisation (age band / city level).

Stronger protection

- k-anonymity: every record is indistinguishable from at least $k-1$ others;
- differential privacy: add noise to statistical results and quote a privacy budget ε;
- synthetic data: keep the statistical properties, keep no real individual.

Compliance bottom line (consistent with the compliance red lines in L2): personal information must be minimal and necessary and purpose-limited, and un-de-identified data must never go into a public repository; cross-border transfer and redistribution need a separate assessment.

Source: PII handling and privacy-enhancing techniques compiled from PIPL/GDPR requirements and privacy-engineering practice.

Drift Monitoring and Rollback

Two kinds of drift

- Schema drift: a column is renamed or deleted, a type changes, an enumeration gains a member → assertions fail immediately;
- Distribution drift: the missing rate, the mean or the category shares change slowly → you need monitoring metrics.

What to monitor

missing rate · duplicate rate of the unique key · numeric quantiles · category shares · PSI / KL divergence · upstream API success rate and latency. Give every metric a threshold and an action (alert / block the release).

Rollback

snapshot + versioned scripts = one command back to the previous version; the analysis table can be rebuilt without re-collecting data.

How it connects to the lecture

"immutable snapshots + idempotent writes" on frame 14 are the precondition for rollback.

Source: Drift metrics and the rollback process are compiled from data-engineering practice; they link to data pipelines in L1 and to MLOps monitoring in L5.

Summary: Preparation as Engineering

One pipeline

contract → assertion → log → monitoring → rollback: every time you add a data source, complete these five things again instead of saying "let's just handle it by hand this time".

Three levels of maturity

1. It runs: scripted and re-runnable;
2. It can be trusted: assertions + quality report + audit log;
3. It can be operated: versioned contracts, drift monitoring, automatic rollback.

The goal of the course project is to make levels 1–2 solid: scripts + assertions + report; level 3 earns extra credit in the project.

Project 1 Requirements

Deliverables (four)

1. Preparation script: from raw data to the analysis table, re-runnable in one command (with assertions and logs);
2. the cleaned dataset: raw / curated layers, including marker columns such as `filled_flag`;
3. data documentation: field dictionary, units, definitions, missing-value and imputation treatment;
4. quality report: five-dimension score + treatment list + impact statement (template on frame 86).

Marking dimensions

understanding the requirement · standard of data handling · reproducibility (can someone else re-run it) · interpretation of results · quality of the report · AI-use declaration. (100 marks in total.)

Common ways to lose marks: no log, missing-value treatment not explained, figures that do not match the conclusions, a script that will not run on someone else's machine.

Source: Consistent with the Project section of Section 6 of the syllabus; teams of at most 2, free choice of data, and use of AI tools is encouraged (declare and verify).

One-Page Proposal (submit by Week 6)

Element	What to write
Question	a one-sentence research question + why it matters (business, finance, trade or marketing context)
Data	source, coverage, time span, how it is obtained and licensed; known quality problems
Method	the preparation pipeline (combine / clean / missing / transform) + the planned analysis or model
Output	the list of deliverables (script, dataset, report) + the expected shape of the conclusion
Risks	risks such as data being unavailable, inconsistent definitions or too small a sample, and the fallback plan

Writing advice: one A4 page, five paragraphs, 3–5 sentences each; making the "Data" paragraph clear matters more than making the "Method" paragraph look elegant.

Source: Consistent with the syllabus requirement to submit a one-page proposal by Week 6.

Quality Report Template

Part 1: the five-dimension score

validity / accuracy / completeness / consistency / uniformity of units (or uniqueness) – for each, give a score + the definition used + the worst field.

Part 2: the treatment list

one line per step: step · rows affected · grounds · owner (this is `l3_cleaning_log.csv`).

Part 3: quantified conclusions

missing rate and pattern · duplicate rate · outlier rate and treatment · number of category variants before and after merging · impact of unit conversion.

Part 4: impact and limitations

the potential effect of the treatments on the conclusions, unresolved issues and the next steps.

One page is enough; the key is that every statement is followed by a number, not by adjectives such as "the data quality is rather good".

Source: The template is compiled from the outputs of the sample and code frames of this lecture (frames 15, 41, 46, 52).

Cleaning and Preparation Checklist (30 items)

Reading and structure

☐ specify the encoding and dtype; ☐ keep the raw snapshot; ☐ state the grain (what one row represents); ☐ assert primary-key uniqueness; ☐ the time column parses; ☐ raw/curated directory layers

Combine and reshape

☐ schema alignment (names/types/units); ☐ declare the cardinality with validate; ☐ reconcile with indicator; ☐ row-count conservation; ☐ the `source` marker; ☐ lossless wide/long round trips

Cleaning

☐ exact duplicates; ☐ a rule for duplicate primary keys; ☐ irrelevant observations; ☐ the category mapping table; ☐ dirty text; ☐ classify outliers (impossible vs statistical)

Missing

☐ missing rate and pattern; ☐ determine the mechanism (MCAR/MAR/MNAR); ☐ the strategy and its grounds; ☐ flag the imputed values; ☐ sensitivity analysis

Panel

☐ $(id, time)$ unique; ☐ gaps and T_i; ☐ why individuals enter and leave; ☐ the coverage table

Transformation and delivery

☐ uniform units; ☐ distribution and standardisation; ☐ field dictionary; ☐ all assertions pass; ☐ quality report; ☐ AI-use declaration

Source: The checklist is compiled from the seven parts of this lecture and can be used directly as a self-check list for Project 1.

Toolbox

Task	Tool / key API
In-memory preparation	pandas: `merge` / `concat` / `pivot` / `melt` / `groupby` / `fillna` / `interpolate`
Large-table preparation	Polars (LazyFrame) · DuckDB (SQL over Parquet) · writing Parquet
Contracts and validation	pandera · Great Expectations · hand-written assertions (frame 74 of this lecture)
Engineering transformation	dbt (models + tests + docs + lineage) · feature store (Feast)
Interactive cleaning	OpenRefine (small data, manual category review) · Jupyter + pandas
Panels and statistics	statsmodels / linearmodels (panel fixed effects) · Stata (`xtset`/`ipolate`)
Monitoring	Evidently / whylogs (distribution drift) · scheduling alerts (Airflow / Dagster)

Source: Tools compiled from the "Tools and data" column of Lecture 3 in the syllabus (pandas, Polars, DuckDB, OpenRefine, SQL) and from 2026 engineering practice.

Glossaries

Term	Meaning in this lecture
Validity	the data satisfy the defined business rules
Accuracy	closeness to the true value
Completeness	all required data are known
Consistency	agreement within and across datasets
Uniformity / Uniqueness	one unit everywhere / a unique primary key
Grain	what one row of the table represents
Append / Merge	add observations / add variables
Cardinality	the 1:1, 1:m, m:1, m:m relation of keys
Reshaping	change the layout, not the content

Term	Meaning in this lecture
MCAR / MAR / MNAR	the three missingness mechanisms
Imputation / Multiple imputation	fill values in / several filled datasets combined
Interpolation / Extrapolation	between known points / beyond the known range
Balanced panel	the same T for every individual
Coverage rate	observed periods / periods that should exist
Winsorise	keep the row, pull the value to a bound
Data contract	field dictionary + types + domains + SLA
Drift	schema or distribution change over time
Rollback	return to the previous version in one command

Source: Terms taken from the body of this lecture and the wording of the original deck, so that you can read the English literature and documentation.

References

Data preparation and cleaning

- McKinney, W. *Python for Data Analysis*, 3rd ed., Ch.7 Data Cleaning and Preparation, Ch.8 Data Wrangling: Join, Combine, Reshape (the set textbook, corresponding to the L3 reading in the syllabus)
- pandas User Guide: *Merge, join, concatenate and compare*, *Reshaping and pivot tables*, *Working with missing data*
- Rahm & Do (2000) Data cleaning: problems and current approaches; Kim et al. (2003) taxonomy of dirty data

Missing data and panels

- Little & Rubin, *Statistical Analysis with Missing Data*
- van Buuren, *Flexible Imputation of Missing Data*
- Enders, *Applied Missing Data Analysis*
- Stata manuals: `merge` / `append` / `joinby` / `xtset` / `ipolate` (including the official examples gymdata and ipolxmpl1)

Engineering documentation: pandera · Great Expectations · dbt · Feast · Polars · DuckDB · Evidently (all online documentation, 2026).

Source: Textbooks and papers correspond to the "Tools and data" and reading requirements for L3 in the syllabus; the engineering documentation is the 2026 version.

Reproducibility Drill

Class exercise (15 minutes)

1. swap script + raw data with no verbal explanation;
2. your partner re-runs it on their own computer;
3. compare: row count, column count, the mean and missing rate of key columns;
4. if they differ → find which step depended on implicit information (a local file, a manual edit, an un-fixed order).

Common causes of "reproduction failure"

a random seed that is not fixed · a dependency on a local file path · data edited by hand in Excel · dependencies not pinned · reliance on dictionary order (use column-name access instead) · time-zone or encoding differences

Every program in this lecture passes this test: `python run_all.py` produces exactly the same samples and figures on any machine.

Source: The drill is designed to match the "five rules for reproducible preparation" of this lecture (frame 14) and the marking dimensions of Project 1.

Thank you!

Questions & discussion

Summary of this lecture: five quality characteristics → combine and reshape → the five cleaning steps → missingness mechanisms and imputation → panel balance → transformation and engineering

Samples and programs: `website/code/lecture03` (10 programs, line-by-line comments)
`website/data/lecture03` (39 samples) `website/figures/lecture03` (12 figures)

Lecturer: Xieer Dai (David) · xieer.dai@szu.edu.cn · Sifang Building South 207